Project Report
On
Database Implementation in JDBC
(CSE 3488)

Library Loan Management System Using JDBC and Apache Derby

Submitted by:

Name:	**Taniprava Sahoo**
Registration No:	**2341019169**
Branch:	**B. Tech. Computer Science & Engineering [AIML]**
Semester:	**6th**
Section:	**23412A1**

INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY),
BHUBANESWAR, ODISHA

Abstract

The Library Loan Management System is a console-based database application developed using Java JDBC and Apache Derby database. The project is designed to automate and simplify basic library operations such as member registration, book management, loan processing, return handling, and record viewing through a menu-driven interface. The system demonstrates the practical implementation of database connectivity, SQL operations, transaction management, and performance evaluation techniques in Java.

The application establishes database connectivity using JDBC and performs database operations efficiently through SQL queries and `*PreparedStatement*`. Separate modules are used for handling database connection, schema initialization, business operations, transaction processing, and performance benchmarking. The project follows a modular structure that improves code readability, maintainability, and execution efficiency.

A major focus of this project is transaction management. Loan processing operations are implemented using explicit transaction handling techniques such as commit, rollback, auto-commit control, and savepoints to maintain data consistency and integrity. The system ensures that incomplete or invalid operations do not affect the database state, thereby demonstrating important ACID properties of database systems.

The project also includes CRUD operations for managing books, members, and loan records. Users can add and view records, process book loans, and return books through the console application. In addition, the project contains a performance evaluation module that compares normal insert operations with batch insert techniques. Benchmarking results show that batch processing significantly improves execution speed and reduces database communication overhead.

Overall, the project provides practical exposure to Java database programming concepts including JDBC connectivity, transaction management, schema creation, SQL query execution, exception handling, and database performance optimization using Apache Derby database.

Contents

Objective

The main objective of this project is to design and develop a Library Loan Management System using Java JDBC and Apache Derby database. The system is developed to automate basic library operations such as managing books, registering members, issuing books, returning books, and maintaining loan records through a menu-driven console application.

This project aims to provide practical understanding of JDBC connectivity and demonstrate how Java applications interact with relational databases. It also focuses on implementing transaction management using commit and rollback mechanisms to maintain data consistency and integrity during database operations. The project demonstrates the importance of ACID properties in database systems.

Another important objective of the project is to evaluate the performance of different JDBC strategies such as normal insert operations and batch insert operations. The benchmarking module helps in understanding how optimized database operations improve execution efficiency.

Main Objectives

• To develop a Library Loan Management System using Java and Apache Derby database.

• To establish database connectivity using JDBC.

• To implement CRUD operations for books, members, and loans.

• To implement transaction management using commit and rollback operations.

• To maintain database consistency using ACID properties.

• To compare the performance of normal insert and batch insert operations.

• To create a menu-driven console application for user interaction.

Problem Statement

Traditional library management systems often face issues such as improper record handling, inefficient transaction processing, and difficulties in maintaining database consistency during multiple operations. Manual management of books, members, and loan records can lead to errors, duplicate entries, data inconsistency, and inefficient performance.

The objective of this project is to develop a Library Loan Management System using Java JDBC and Apache Derby database that can efficiently manage library operations while ensuring secure and reliable transaction handling. The system is designed to automate important library functionalities such as member registration, book management, loan processing, return handling, and record viewing through a menu-driven console application.

The project also focuses on implementing transaction management techniques to maintain data integrity during database operations. In addition, the system evaluates the performance of different JDBC strategies to identify more efficient methods of database execution.

Major Problems Addressed

- Difficulty in maintaining book and member records manually.
- Risk of inconsistent data during loan processing operations.
- Lack of proper transaction handling in traditional systems.
- Inefficient execution of repeated database operations.
- Poor performance due to non-optimized JDBC operations.
- Need for secure and reliable database connectivity.
- Lack of automated management for library loan activities.

Project Detail

The Library Loan Management System is a JDBC-based application developed using Java and Apache Derby database. The project automates library operations while demonstrating database connectivity, transaction management, CRUD operations, benchmarking, and menu-driven execution.

a. Introduction

The Library Loan Management System is a console-based database application developed using Java JDBC and Apache Derby database. The system is designed to automate and simplify various library operations such as adding books, registering members, processing book loans, returning books, and viewing records. The application uses JDBC connectivity to interact with the database and execute SQL operations efficiently.

The project demonstrates the practical implementation of database connectivity, transaction management, CRUD operations, and performance evaluation techniques. The system maintains proper data consistency by using transaction management concepts such as commit and rollback during loan processing operations.

b. Modules Used

The system is divided into different modules, where each module performs a specific functionality of the application.

Table 3.1 Modules and Their Functionalities

Module	Description
ConnectionManager	Establishes and manages database connections
DatabaseSetup	Creates database tables and initializes schema
BusinessLogic	Performs CRUD operations for books and members
TransactionService	Handles loan transactions using commit and rollback
PerformanceEvaluator	Performs benchmarking and performance testing

c. Database Tables

The database consists of multiple tables designed to store and manage members, books, and loan records efficiently.

Table 3.2 Members Table

Column Name	Data Type
MemberID	INT
Name	VARCHAR
ActiveLoans	INT

Table 3.2 Books Table

Column Name	Data Type
BookID	INT
Title	VARCHAR
ISBN	VARCHAR
Available	BOOLEAN

Table 3.2 Loans Table

Column Name	Data Type
LoanID	INT
MemberID	INT
BookID	INT
LoanDate	DATE
ReturnDate	DATE

d. Functionalities of the System

The application provides the following functionalities:

- Add new library members.
- Add new books into the database.
- Display available books and member records.
- Process book loan transactions.
- Return issued books.
- Display loan details.
- Perform JDBC performance benchmarking.
- Compare normal insert and batch insert execution times

e. Workflow of the System

The following steps describe the overall workflow and execution process of the Library Loan Management System:

1. User selects an operation from the menu-driven console application.
2. JDBC establishes connection with Apache Derby database.
3. SQL queries execute using *PreparedStatement.*
4. Transaction operations are committed or rolled back depending on execution status.
5. Results and outputs are displayed to the user through the console.

f. Transaction Management

Transaction management was implemented using:

- setAutoCommit(false)
- commit()
- rollback()
- savepoints

These operations ensure database consistency during loan processing and prevent partial or invalid updates in case of failures.

g. Performance Evaluation

The project includes a benchmarking module to compare the execution performance of:

- Normal insert operations using *executeUpdate()*
- Batch insert operations using *addBatch()* and *executeBatch()*

The results showed that batch insert operations are significantly faster and more efficient compared to normal insert execution.

Planning

The development of the Library Loan Management System was carried out in multiple phases to ensure proper implementation, testing, and performance evaluation of the application. A step-by-step planning approach was followed to organize the project efficiently and maintain proper workflow during development.

Initially, the project requirements and objectives were analyzed to understand the functionalities needed in the system. After requirement analysis, the database schema was designed by identifying the required tables, relationships, and constraints for managing books, members, and loan records.

The next stage involved establishing JDBC connectivity between the Java application and Apache Derby database. After successful database connection, CRUD operations for books, members, and loans were implemented. Transaction management features such as commit, rollback, and savepoints were then added to maintain data consistency and integrity during loan processing operations.

Once the core functionalities were completed, a menu-driven console interface was developed to allow user interaction with the system. Finally, benchmarking and performance evaluation modules were implemented to compare different JDBC execution strategies and analyze system performance.

Project Planning Phases:

1. Requirement analysis and project objective identification.
2. Database schema design and table creation.
3. JDBC connectivity implementation.
4. Development of CRUD operations.
5. Implementation of transaction management.
6. Creation of menu-driven console application.
7. Performance benchmarking and testing.
8. Validation, debugging, and result analysis.

Hardware and Software Specifications

Hardware Requirements:

The following hardware components were used for the development and execution of the project:

Table 5.1: Hardware Requirements

Hardware Component	Specification
Processor	Intel Core i5 / i7
RAM	8 GB
Storage	256 GB SSD
Keyboard	Standard Keyboard
Monitor	15.6-inch Display

Software Requirements:

The following software tools and technologies were used during the development of the project:

Table 5.2: Software Requirements

Software	Description
Operating System	Windows 10 / Windows 11
Programming Language	Java
IDE	Eclipse IDE
Database	Apache Derby
Connectivity	JDBC (Java Database Connectivity)
JDK Version	Java SE 21

Technologies Used:

The following technologies were used for the development and execution of the Library Loan Management System:

• Java for application development.

• JDBC for database connectivity.

• Apache Derby as embedded relational database.

• SQL for database operations and queries.

• Eclipse IDE for coding, debugging, and project execution.

Implementations

The implementation of the Library Loan Management System was carried out using Java JDBC and Apache Derby database. The application was developed in a modular manner where each class was responsible for a specific functionality such as database connection, transaction management, CRUD operations, benchmarking, and menu-driven execution.

The project uses JDBC connectivity to establish communication between the Java application and the Derby database. SQL queries were executed using *PreparedStatement* to improve performance and security. Transaction management was implemented using commit and rollback operations to maintain database consistency during loan process.

6.1 Database Connection Implementation

The *ConnectionManager.java* class was created to establish and manage the connection between Java application and Apache Derby database.

Code Snippet:

```
package com.dbms.project_2341019169;

import java.sql.Connection;

public class ConnectionManager {

    private static final String URL =
            "jdbc:derby:libraryDB;create=true";

    public static Connection getConnection()
            throws SQLException {

        return DriverManager.getConnection(URL);
    }

    public static void shutdown() {

        try {

            DriverManager.getConnection(
                    "jdbc:derby:libraryDB;shutdown=true");

        } catch (SQLException e) {

            System.out.println(
                    "Database shutdown successful.");
        }
    }
}
```

Explanation:

The *ConnectionManager* class manages the database connection lifecycle. It establishes connectivity with Apache Derby database using JDBC and also provides database shutdown functionality.

6.2 Database Initialization and Table Creation

The *DatabaseSetup.java* class was implemented to create the required database tables such as Members, Books, and Loans.

Primary keys, foreign keys, and constraints were used to maintain proper relationships between tables.

Code Snippet:

```
package com.dbms.project_2341019169;

import java.sql.Connection;

public class DatabaseSetup {

    public static void initializeDatabase() {

        try {
                Connection con =
                        ConnectionManager.getConnection();

                Statement stmt =
                        con.createStatement()
        ) {

            stmt.executeUpdate(
                    "CREATE TABLE Members (" +
                    "MemberID INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY," +
                    "Name VARCHAR(100), " +
                    "ActiveLoans INT DEFAULT 0)"
            );

            stmt.executeUpdate(
                    "CREATE TABLE Books (" +
                    "BookID INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY," +
                    "Title VARCHAR(200), " +
                    "ISBN VARCHAR(50), " +
                    "Available BOOLEAN)"
            );

            stmt.executeUpdate(
                    "CREATE TABLE Loans (" +
                    "LoanID INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY, " +
                    "MemberID INT, " +
                    "BookID INT, " +
                    "LoanDate DATE, " +
                    "ReturnDate DATE, " +
                    "FOREIGN KEY (MemberID) REFERENCES Members(MemberID), " +
                    "FOREIGN KEY (BookID) REFERENCES Books(BookID))"
            );

            System.out.println(
                    "Database initialized successfully.");

        } catch (Exception e) {
            System.out.println(
                    "Tables may already exist.");
        }
    }
}
```

Explanation:

The *DatabaseSetup* class initializes the database schema and creates the required tables. The system uses relational database concepts with foreign key constraints to maintain consistency between members, books, and loan records.

6.3 Transaction Management Implementation

The *TransactionService.java* class was developed to process loan transactions using transaction management techniques such as commit, rollback, and savepoints.

The system first checks whether a book is available. If available, the book status is updated, loan record is inserted, and active loan count is updated. If any error occurs, rollback restores the database to its previous consistent state.

Code Snippet:

```
package com.dbms.project_2341019169;

import java.sql.Connection;

public class TransactionService {

    public static void processLoan(
          int memberId,
          int bookId) {

        Connection con = null;

        try {

            con =
                    ConnectionManager.getConnection();

            /*
             * Disable auto commit
             * to manually control transaction
             */
            con.setAutoCommit(false);

            /*
             * Step 1:
             * Check whether book is available
             */
            String checkQuery =
                    "SELECT Available FROM Books " +
                    "WHERE BookID=?";

            PreparedStatement checkStmt =
                    con.prepareStatement(checkQuery);

            checkStmt.setInt(1, bookId);

            ResultSet rs =
                    checkStmt.executeQuery();

            if (rs.next()) {

                boolean available =
                        rs.getBoolean("Available");

                /*
                 * If book already issued
                 * rollback transaction
                 */
                if (!available) {

                    System.out.println(
                            "Book not available.");
                    con.rollback();
                    return;
                }
            }
```

```
             * Step 2:
             * Update book availability
             */
            String updateBook =
                    "UPDATE Books " +
                    "SET Available=false " +
                    "WHERE BookID=?";

            PreparedStatement ps1 =
                    con.prepareStatement(updateBook);

            ps1.setInt(1, bookId);

            ps1.executeUpdate();

            /*
             * Create savepoint
             */
            Savepoint sp =
                    con.setSavepoint();

            /*
             * Step 3:
             * Insert loan record
             */
            String insertLoan =
                    "INSERT INTO Loans(" +
                    "MemberID, BookID, LoanDate) " +
                    "VALUES(?,?,CURRENT_DATE)";

            PreparedStatement ps2 =
                    con.prepareStatement(insertLoan);

            ps2.setInt(1, memberId);
            ps2.setInt(2, bookId);

            ps2.executeUpdate();

            /*
             * Step 4:
             * Increase member active loan count
             */
            String updateMember =
                    "UPDATE Members " +
                    "SET ActiveLoans = ActiveLoans + 1 " +
                    "WHERE MemberID=?";

            PreparedStatement ps3 =
                    con.prepareStatement(updateMember);

            ps3.setInt(1, memberId);

            ps3.executeUpdate();

            /*
             * Commit transaction
             */
            con.commit();

            System.out.println(
                    "Loan processed successfully.");
```

```
        } catch (Exception e) {

            try {

                /*
                 * Rollback if any failure occurs
                 */
                if (con != null) {

                    con.rollback();

                    System.out.println(
                            "Transaction rolled back.");
                }

            } catch (Exception ex) {

                ex.printStackTrace();
            }

            e.printStackTrace();
        }
    }
}
```

Explanation:

Transaction management was implemented using:

• *setAutoCommit(false)*

• *commit()*

• *rollback()*

• *savepoints*

These operations ensure that all loan-related operations execute successfully together. If any step fails, the transaction is rolled back to maintain database consistency and integrity.

6.4 CRUD Operations Implementation

The *BusinessLogic.java* class was implemented to perform CRUD operations such as adding books, adding members, displaying records, returning books, and viewing loan details.

PreparedStatement was used for executing SQL queries securely and efficiently.

Code Snippet:

```
package com.dbms.project_2341019169;

import java.sql.Connection;

public class BusinessLogic {

    /*
     * Add new member
     */
    public static void addMember(String name) {

        String sql =
                "INSERT INTO Members(Name, ActiveLoans) " +
                "VALUES(?,0)";

        try {
                Connection con =
                        ConnectionManager.getConnection();

                PreparedStatement ps =
                        con.prepareStatement(sql)
        ) {

            ps.setString(1, name);

            ps.executeUpdate();

            System.out.println(
                    "Member added successfully.");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    /*
     * Add new book
     */
    public static void addBook(
            String title,
            String isbn) {
        String sql =
                "INSERT INTO Books(Title, ISBN, Available) " +
                "VALUES(?,?,true)";

        try {
                Connection con =
                        ConnectionManager.getConnection();

                PreparedStatement ps =
                        con.prepareStatement(sql)
        ) {
            ps.setString(1, title);
            ps.setString(2, isbn);

            ps.executeUpdate();

            System.out.println(
                    "Book added successfully.");
        }
    }
```

```
        } catch (Exception e) {

            e.printStackTrace();
        }
    }
    /*
     * Display all books
     */
    public static void showBooks() {

        String sql =
                "SELECT * FROM Books";

        try (
                Connection con =
                        ConnectionManager.getConnection();

                PreparedStatement ps =
                        con.prepareStatement(sql);

                java.sql.ResultSet rs =
                        ps.executeQuery()
        ) {

            System.out.println(
                    "\n===== BOOK LIST =====");

            while (rs.next()) {

                System.out.println(
                        "Book ID : "
                        + rs.getInt("BookID"));

                System.out.println(
                        "Title   : "
                        + rs.getString("Title"));

                System.out.println(
                        "ISBN    : "
                        + rs.getString("ISBN"));

                System.out.println(
                        "Available : "
                        + rs.getBoolean("Available"));

                System.out.println(
                        "----------------------");
            }

        } catch (Exception e) {

            e.printStackTrace();
        }
    }
    /*
     * Return issued book
     */
    public static void returnBook(int bookId) {

        try (
                Connection con =
                        ConnectionManager.getConnection()
        ) {
```

```
        /*
         * Make book available again
         */
        String updateBook =
                "UPDATE Books " +
                "SET Available=true " +
                "WHERE BookID=?";

        PreparedStatement ps1 =
                con.prepareStatement(updateBook);

        ps1.setInt(1, bookId);

        ps1.executeUpdate();

        System.out.println(
                "Book returned successfully.");

    } catch (Exception e) {

        e.printStackTrace();
    }
}
/*
 * Display all members
 */
public static void showMembers() {

    String sql =
            "SELECT * FROM Members";

    try (
            Connection con =
                    ConnectionManager.getConnection();

            PreparedStatement ps =
                    con.prepareStatement(sql);

            java.sql.ResultSet rs =
                    ps.executeQuery()
    ) {

        System.out.println(
                "\n===== MEMBER LIST =====");

        while (rs.next()) {

            System.out.println(
                    "Member ID : "
                    + rs.getInt("MemberID"));

            System.out.println(
                    "Name      : "
                    + rs.getString("Name"));

            System.out.println(
                    "Active Loans : "
                    + rs.getInt("ActiveLoans"));

            System.out.println(
                    "----------------------");
        }
```

```
    } catch (Exception e) {

        e.printStackTrace();
    }
}
/*
 * Display all loans
 */
public static void showLoans() {

    String sql =
            "SELECT * FROM Loans";

    try (
            Connection con =
                    ConnectionManager.getConnection();

            PreparedStatement ps =
                    con.prepareStatement(sql);

            java.sql.ResultSet rs =
                    ps.executeQuery()
    ) {

        System.out.println(
                "\n===== LOAN LIST =====");

        while (rs.next()) {

            System.out.println(
                    "Loan ID : "
                    + rs.getInt("LoanID"));

            System.out.println(
                    "Member ID : "
                    + rs.getInt("MemberID"));

            System.out.println(
                    "Book ID : "
                    + rs.getInt("BookID"));

            System.out.println(
                    "Loan Date : "
                    + rs.getDate("LoanDate"));

            System.out.println(
                    "----------------------");
        }

    } catch (Exception e) {

        e.printStackTrace();
    }
}
}
```

Explanation:

The *BusinessLogic* class handles all CRUD operations of the system. It provides functionalities for member registration, book addition, displaying records, book return handling, and loan record management.

6.5 Menu-Driven Console Application

The *MainApp.java* class was implemented to provide a menu-driven interface for user interaction. Users can select different operations such as adding members, adding books, processing loans, returning books, viewing records, and running benchmarks.

Code Snippet:

```
package com.dbms.project_2341019169;

public class MainApp {

    public static void main(String[] args) {

        DatabaseSetup.initializeDatabase();

        java.util.Scanner sc =
                new java.util.Scanner(System.in);

        int choice;
```

```
do {

    System.out.println(
            "\n===== LIBRARY MENU =====");

    System.out.println(
            "1. Add Member");

    System.out.println(
            "2. Add Book");

    System.out.println(
            "3. Show Books");

    System.out.println(
            "4. Process Loan");

    System.out.println(
            "5. Return Book");
    System.out.println(
            "6. Run Benchmark");
    System.out.println(
            "7. Show Members");
    System.out.println(
            "8. Show Loans");

    System.out.println(
            "0. Exit");

    System.out.print(
            "Enter Choice: ");

    choice = sc.nextInt();

    sc.nextLine();
```

```
switch (choice) {

    case 1:

        System.out.print(
                "Enter Member Name: ");

        String memberName =
                sc.nextLine();

        BusinessLogic.addMember(
                memberName);

        break;

    case 2:

        System.out.print(
                "Enter Book Title: ");

        String title =
                sc.nextLine();

        System.out.print(
                "Enter ISBN: ");

        String isbn =
                sc.nextLine();

        BusinessLogic.addBook(
                title,
                isbn);

        break;

    case 3:

        BusinessLogic.showBooks();

        break;
```

```
case 4:

    System.out.print(
            "Enter Member ID: ");

    int memberId =
            sc.nextInt();

    System.out.print(
            "Enter Book ID: ");

    int bookId =
            sc.nextInt();

    TransactionService.processLoan(
            memberId,
            bookId);

    break;

case 5:

    System.out.print(
            "Enter Book ID: ");

    int returnBookId =
            sc.nextInt();

    BusinessLogic.returnBook(
            returnBookId);

    break;
case 6:

    PerformanceEvaluator.runBenchmarks();

    break;
```

```
            case 7:

                BusinessLogic.showMembers();

                break;
            case 8:

                BusinessLogic.showLoans();

                break;

            case 0:

                ConnectionManager.shutdown();

                System.out.println(
                        "Exiting application.");

                break;

            default:

                System.out.println(
                        "Invalid choice.");
        }

    } while (choice != 0);
    }
}
```

Explanation:

The *MainApp* class acts as the entry point of the application. It provides a console-based menu system that allows users to interact with various functionalities of the Library Loan Management System.

6.6 Performance Benchmarking Implementation

The *PerformanceEvaluator.java* class was developed to compare the execution performance of normal insert operations and batch insert operations.

Execution time was measured using *System.nanoTime()* to evaluate JDBC performance optimization.

Code Snippet:

```
package com.dbms.project_2341019169;

import java.sql.Connection;

public class PerformanceEvaluator {

    /*
     * Run all benchmarks
     */
    public static void runBenchmarks() {

        System.out.println(
                "\n===== PERFORMANCE TEST =====");

        normalInsertBenchmark();

        batchInsertBenchmark();
    }

    /*
     * Normal insert benchmark
     */
    public static void normalInsertBenchmark() {

        Connection con = null;

        try {
            con =
                    ConnectionManager.getConnection();

            con.setAutoCommit(false);

            String sql =
                    "INSERT INTO Members(Name, ActiveLoans) " +
                    "VALUES(?,0)";

            PreparedStatement ps =
                    con.prepareStatement(sql);

            long start =
                    System.nanoTime();

            for (int i = 1; i <= 1000; i++) {
                ps.setString(
                        1,
                        "NormalUser" + i);
                ps.executeUpdate();
            }

            con.commit();

            long end =
                    System.nanoTime();

            double timeMs =
                    (end - start) / 1000000.0;

            System.out.println(
                    "\nNORMAL INSERT:");

            System.out.println(
                    "1000 Records Inserted");

            System.out.println(
                    "Execution Time: "
                    + timeMs
                    + " ms");

        } catch (Exception e) {

            e.printStackTrace();

        } finally {

            try {

                if (con != null) {

                    con.close();
                }

            } catch (Exception ex) {

                ex.printStackTrace();
            }
        }
    }
}
```

```
    /*
     * Batch insert benchmark
     */
    public static void batchInsertBenchmark() {

        Connection con = null;

        try {

            con =
                    ConnectionManager.getConnection();

            con.setAutoCommit(false);

            String sql =
                    "INSERT INTO Members(Name, ActiveLoans) " +
                    "VALUES(?,0)";

            PreparedStatement ps =
                    con.prepareStatement(sql);

            long start =
                    System.nanoTime();

            for (int i = 1; i <= 1000; i++) {

                ps.setString(
                        1,
                        "BatchUser" + i);

                ps.addBatch();
            }

            ps.executeBatch();

            con.commit();

            long end =
                    System.nanoTime();

            double timeMs =
                    (end - start) / 1000000.0;

            System.out.println(
                    "\nBATCH INSERT:");

            System.out.println(
                    "1000 Records Inserted");

            System.out.println(
                    "Execution Time: "
                    + timeMs
                    + " ms");

        } catch (Exception e) {

            e.printStackTrace();

        } finally {

            try {

                if (con != null) {

                    con.close();
                }

            } catch (Exception ex) {

                ex.printStackTrace();
            }
        }
    }
}
```

```
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
                if (con != null) {
                    con.close();
                }
            } catch (Exception ex) {
                ex.printStackTrace();
            }
        }
    }
```

Explanation:

The benchmarking module compares normal insert operations with batch insert operations. Results showed that batch insert execution is significantly faster because multiple SQL statements are executed together, reducing database communication overhead.

6.7 Overall System Architecture

The project was developed using modular architecture where each Java class performs a separate responsibility. This improves code readability, maintainability, and scalability.

Table 3.3: Classes and Their Responsibilities

Class Name	**Responsibility**
ConnectionManager	Database connection handling
DatabaseSetup	Table creation and schema setup
BusinessLogic	CRUD operations
TransactionService	Transaction management
PerformanceEvaluator	Benchmark evaluation
MainApp	Menu-driven execution

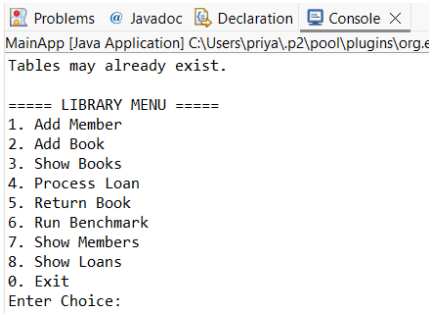
Results and output

The Library Loan Management System was successfully implemented using Java JDBC and Apache Derby database. The application successfully performed all CRUD operations, transaction management operations, and performance benchmarking functionalities through a menu-driven console interface.

The system was tested with different operations such as member registration, book addition, loan processing, return handling, record viewing, and JDBC performance evaluation. All functionalities executed successfully without data inconsistency issues.

The benchmark results showed that batch insert operations performed significantly faster compared to normal insert operations due to optimized execution and reduced database communication overhead.

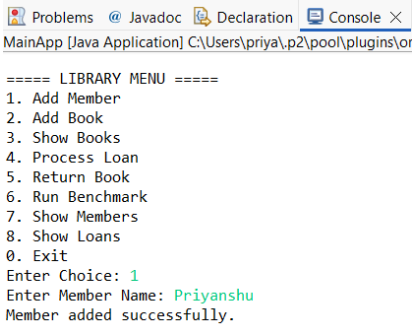
7.1 Main Menu Output



Explanation:

The menu-driven console application allows users to perform various operations such as adding members, adding books, processing loans, returning books, viewing records, and running performance benchmarks.

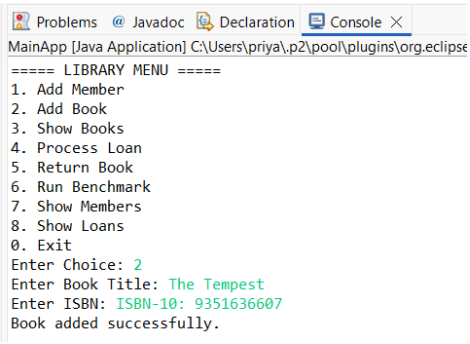
7.2 Add Member Output



Explanation:

The system successfully inserted member records into the database using *PreparedStatement* and JDBC connectivity.

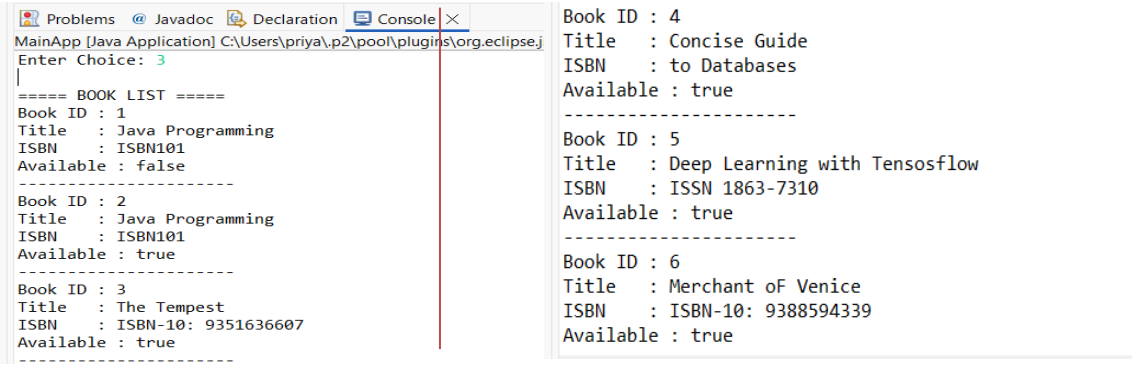
7.3 Add Book Output



Explanation;

Book records were successfully added into the database along with title, ISBN number, and availability status.

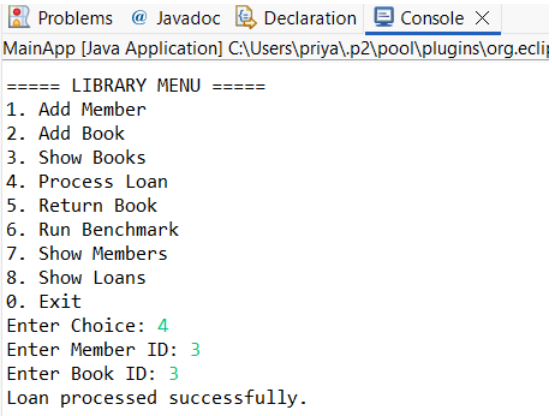
7.4 Show Books Output



Explanation:

The application successfully retrieved and displayed all book records stored in the database.

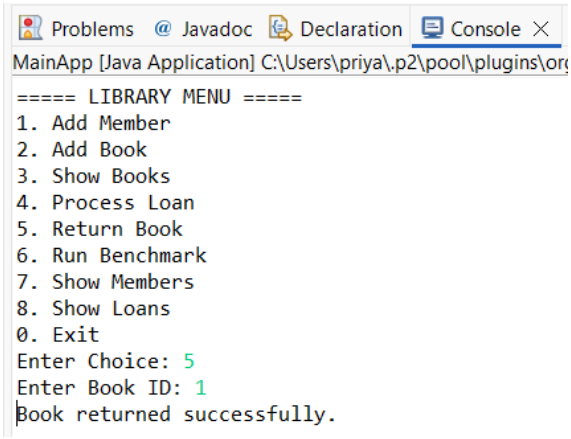
7.5 Loan Processing Output



Explanation:

The transaction management system successfully processed book loan operations using commit and rollback mechanisms to maintain data consistency.

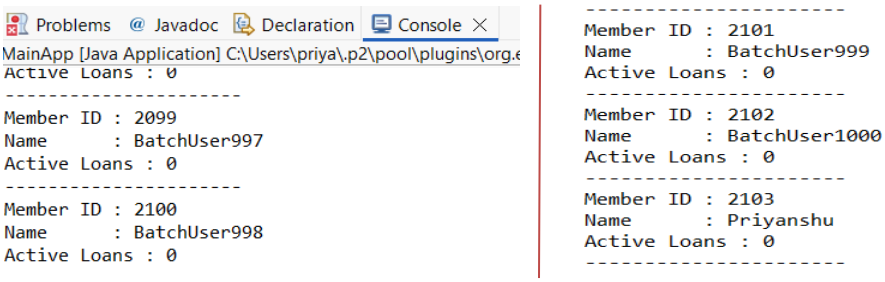
7.6 Return Book Output



Explanation:

The return book functionality successfully updated the availability status of issued books.

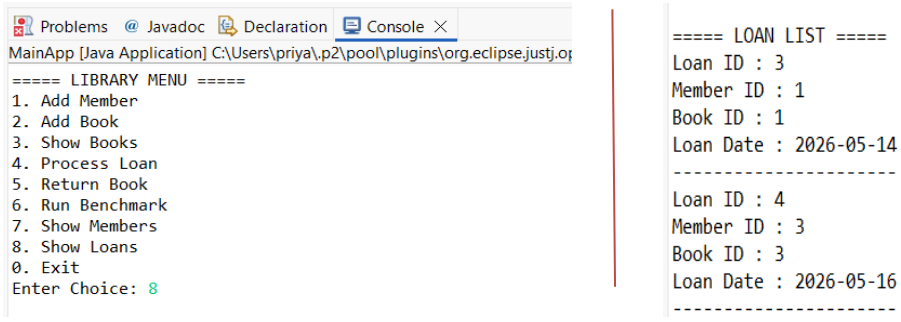
7.7 Show Members Output



Explanation:

The system successfully displayed all registered member records along with active loan details.

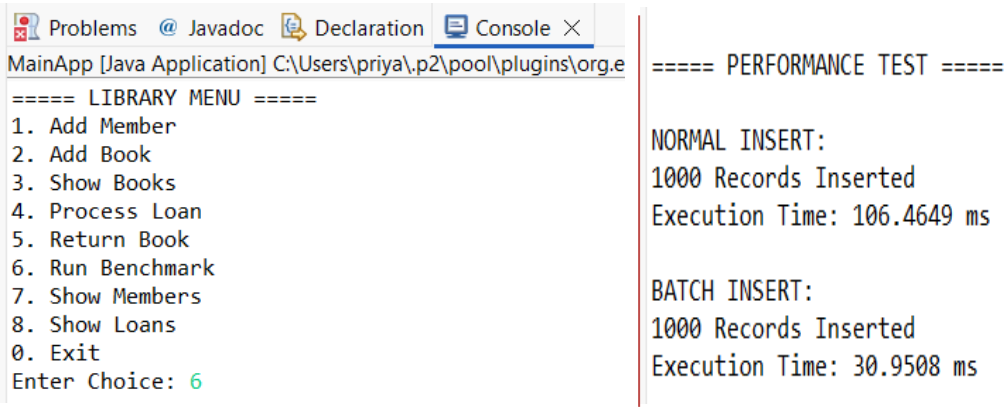
7.8 Show Loans Output



Explanation:

The application successfully displayed all loan transaction records stored in the database.

7.9 Benchmark Output



Explanation:

Performance benchmarking was conducted to compare normal insert operations and batch insert operations. The results showed that batch insert execution was significantly faster and more efficient.

PERFORMANCE COMPARISON TABLE

Table 7.1: Performance Comparison of Insert Operations

Operation	Records Inserted	Execution Time
Normal Insert	1000	106.4649 ms
Batch Insert	1000	30.9508 ms

Obseravations:

The following observations were obtained after testing and evaluating the Library Loan Management System:

• All CRUD operations executed successfully.

• Transaction management maintained database consistency.

• Rollback functionality prevented invalid updates.

• Batch insert operations showed better performance than normal inserts.

• The system successfully demonstrated JDBC connectivity and database optimization techniques.

Conclusion

The Library Loan Management System was successfully developed using Java JDBC and Apache Derby database. The project demonstrated the practical implementation of database connectivity, transaction management, CRUD operations, and performance benchmarking through a menu-driven console application. The system efficiently managed library activities such as member registration, book management, loan processing, return handling, and record viewing.

Transaction management was successfully implemented using commit and rollback operations to maintain database consistency and integrity during loan transactions. The project also demonstrated the importance of ACID properties in database systems and showed how JDBC can be used for secure and efficient interaction with relational databases.

Performance benchmarking was conducted to compare normal insert operations and batch insert operations. The results proved that batch insert execution is significantly faster and more optimized compared to normal insert execution due to reduced database communication overhead.

Key Highlights of the Project

- Successfully established JDBC connectivity with Apache Derby database.
- Implemented CRUD operations for books, members, and loans.
- Implemented transaction management using commit and rollback.
- Maintained database consistency using ACID properties.
- Developed a menu-driven console application for user interaction.
- Performed performance benchmarking and optimization analysis.
- Demonstrated the efficiency of batch insert operations over normal inserts.

Overall, the project provided practical knowledge of JDBC programming, SQL operations, database schema design, transaction handling, and performance optimization techniques. The developed application successfully achieved all planned objectives and demonstrated efficient database application development using Java and Apache Derby.

References

[1] E. Sciore, Database Design and Implementation, Cham, Switzerland: Springer, 2020.

[2] Oracle Corporation, “Java Database Connectivity (JDBC) API,” Oracle Documentation. [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/

[3] Apache Software Foundation, “Apache Derby Documentation,” Apache Derby Official Documentation. [Online]. Available: https://db.apache.org/derby/docs/. [Accessed: 16-May-2026].

[4] P. Deitel and H. Deitel, Java How to Program, 11th ed. Hoboken, NJ, USA: Pearson, 2018.-